241UC2414K_LEW_ENG_JEAN_REVIEW_PAPER.docx

by Lew Eng Jean

Submission date: 29-Jun-2025 12:54PM (UTC+0800)
Submission ID: 2707500215
File name: 241UC2414K_LEW_ENG_JEAN_REVIEW_PAPER.docx (376.96K)
Word count: 3875
Character count: 24927

Applying Machine Learning Techniques to Predict Software Defects

Lew Eng Jean
Faculty of Computing and Informatics,
Multimedia University, MMU
Cyberjaya, Selangor
lew.eng.jean@student.mmu.edu.my

line 1: 2nd Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

line 1: 3rd Given Name Surname
line 2: *dept. name of organization (of Affiliation)*
line 3: *name of organization (of Affiliation)*
line 4: City, Country
line 5: email address or ORCID

***Abstract*— Software Defect Prediction (SDP) plays a pivotal role in increasing software reliability and reducing maintenance by identifying faulty modules before deployment. Traditional machine learning models tend to be prone to class imbalance, noisy metrics, and scalability issues. In this research, an innovative hybrid machine learning framework based on Random Forest (RF) and Support Vector Machine (SVM) augmented with a meta-classifier for improved prediction accuracy and robustness is proposed. RF is employed for ranking feature importance and feature reduction, whereas a fast linear SVM is employed for rapid classification. A Gradient Boosting meta-model is trained on the results of RF and SVM to output final predictions. NASA JM1 dataset was used for experimentation due to its complexity and size. SMOTE and Platt Scaling methods were employed to address class imbalance and improve calibration. Experimental results demonstrate that RF-SVM+ model outperforms single classifiers in terms of high F1-score (0.8364) on the minority class and a good ROC-AUC score (0.9096), but with low run time. Results reflect the efficiency of the model to handle real-world imbalanced datasets and its potential application within automated software quality assurance pipelines.**

Keywords*— *Software Defects Prediction, Machine Learning, Random Forest, Support Vector Machine, Ensemble Learning

I. INTRODUCTION

In the era of rapid digital transformation, software systems have become integral to nearly every domain—from healthcare to finance. Ensuring the reliability and quality of such systems is essential, and Software Defect Prediction (SDP) using machine learning (ML) has emerged as a vital tool in this regard. SDP aims to identify potential defects early in the development process to minimize failures, reduce maintenance costs, and ensure timely delivery. However, real-world implementation of ML-based SDP faces two persistent challenges: class imbalance in datasets and high computational complexity.

Class imbalance, where non-defective instances vastly outnumber defective ones, often leads to models that favor the majority class and overlook critical minority cases. Several studies, including Feng et al. [1] and Dhavileswarapu et al. [2], have shown that this imbalance significantly diminishes predictive accuracy. On the other hand, high-dimensional feature sets and complex models can result in increased computational load, limiting scalability and real-time applicability. To address these issues, researchers have explored techniques like ensemble learning and parallel processing to enhance accuracy and efficiency [3].

This research proposes a hybrid ML model that integrates Random Forest (RF) for feature selection and Support Vector Machine (SVM) for classification, further enhanced by a meta-classifier to improve robustness. The model is evaluated using the NASA JM1 dataset, characterized by high dimensionality and class imbalance. By employing techniques such as SMOTE and Platt Scaling, the study aims to reduce bias, lower runtime, and boost predictive performance.

The overarching goals of this research are to (1) develop a hybrid approach that improves prediction accuracy, (2) reduce computational complexity, and (3) assess the model's effectiveness relative to traditional classifiers. The findings aim to contribute not only to the refinement of SDP strategies but also offer transferable insights for broader applications in machine learning and automated software quality assurance. Literature review

II. LITERATURE REVIEW

A. Class Imbalance in Dataset

Class imbalance remains a major challenge in software defect prediction (SDP), where defective instances are significantly outnumbered by non-defective ones. This imbalance skews models toward majority class predictions, reducing effectiveness (Feng et al., 2021; Dhavileswarapu et al., 2021). To improve early defect detection, researchers have proposed various solutions. Hao Wang et al. (2024) introduced a binary Gray Wolf Optimizer (bGWO) combined with traditional classifiers and oversampling. This approach enhances feature selection and maintains high accuracy while managing computational complexity. Dhavileswarapu et al. (2021) emphasized synthetic oversampling techniques like SMOTE and COSTE, which generate new minority instances to balance the data. They also noted improvements using SMOTE-TUNED and neural networks for more refined handling of imbalance. COSTE, proposed by Feng et al., generates complex-aware synthetic samples and outperforms standard SMOTE variants in detection and false alarm rates. Then, Xin Dong et al. (2023) compared traditional and deep learning models, highlighting that ensemble techniques like Bagging, Boosting, and Stacking achieved the best overall performance, with Boosting being particularly effective in time-sensitive settings. Sharma et al. (2023) reviewed ensemble learning methods like Bagging, Boosting, and Stacking, highlighting the role of under-sampling and metaheuristics in boosting model performance. Similarly, Saharudin et al. (2020) evaluated 31 studies and noted that ensemble frameworks and CK metrics (e.g., LOC, CBO) are common, though challenges like redundancy and irrelevant features persist. Al-Smadi et al. (2023) and Ibrahim et al. (2023) emphasized explainability and feature selection, leveraging SMOTE and nature-inspired algorithms (e.g., PSO, ant colony) to improve class balance and model transparency. Their work also notes trade-offs between recall gains and drops in precision using balanced ensemble

methods. Jumare et al. (2024) compared ML and DL models on the JM1 dataset using SMOTE-Tomek. RF achieved the highest performance (F1-score: 0.898), with CNN showing strong recall, highlighting the effectiveness of ensembled and deep learning combinations for defect detection. Khalid et al. (2023) conducted a systematic review employing clustering and metaheuristics to refine ML performance. Corrected SVM models achieved top accuracy (99.8%), outperforming traditional methods and affirming the potential of optimized and hybrid approaches.

B. *Computational Complexity and Time Consumption*

Mishra and Sharma (2024) proposed a DL-based SDP model for CI/CD pipelines using MSMOTE and F-BERT for feature extraction, and Bi-CGRU for defect prediction, achieving strong accuracy (95.32%) and F1-score (91.36%) with reduced labeling noise. Dong et al. (2023) showed that ensemble learning (Bagging, Boosting, Stacking) outperforms traditional ML and DL models, reaching 99% AUC and 97% F1-score. Boosting was particularly effective in time-sensitive scenarios. Ibrahim et al. (2023) emphasized handling computational complexity by applying balanced ensemble algorithms like Easy Ensemble and Balanced RF. While recall and AUC improved, precision dropped—highlighting trade-offs from re-sampling. Khalid et al. (2023) tackled complexity and imbalance using SMOTE and a wide range of ML models, with XGBR delivering top performance. Explainable AI tools (SHAP, LIME) identified key metrics like static invocations and CBO as impactful predictors. Singh et al. (2024) introduced a CPSDP model using structural code metrics and XGBoost, improving AUC by 7% over prior methods. While efficient, its performance relied heavily on quality training data, making dataset selection a critical step.

III. RESEARCH METHODOLOGY

In this chapter, the approach of solving problems of SDP was presented with the proposed solution methods and the progress achieved thus far. Research methodology serves as the foundation for systematically conducting investigations, guiding the processes throughout the project. The methodology implemented in this research, which is shown in Figure 3.1, includes key components: data collection, data analysis, data preprocessing, hybrid model development, the training and evaluation of the proposed model, validation, and verification.

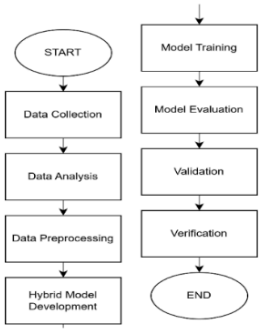


Fig. 1. Research Methodology

The inception of the methodology begins with the data collection phase.

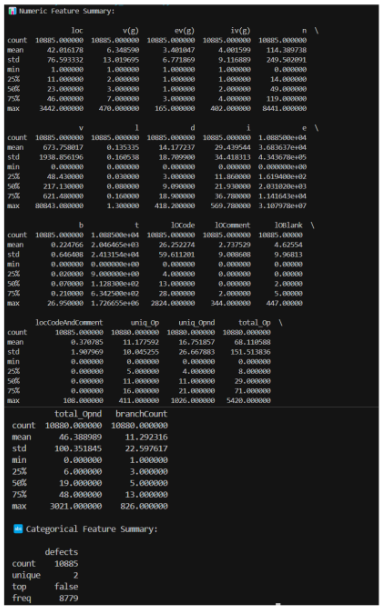
A. *DATA COLLECTION*

The foundation of this research lies in the acquisition of high-quality, representative data relevant to software defect prediction. For this purpose, the JM1 dataset was selected from the publicly available NASA PROMISE dataset repository, a trusted benchmark in empirical software engineering studies. The dataset source is listed in the References section.

B. *DATA ANALYSIS*

To gain an initial understanding of the NASA JM1 dataset, data analysis is performed to summarize the main characteristics of the data.

The summary statistics provide a quick summary of measures of the central tendency, dispersion, and shape of the distribution of the features. Figure 3.5 and 3.6 below showcases the summary statistics for various metrics related to LOC and other software attributes in the dataset.



Numeric Feature Summary:

	loc	v(g)	ev(g)	iv(g)	n
count	10885.000000	10885.000000	10885.000000	10885.000000	10885.000000
mean	42.016178	6.348598	3.401047	4.001599	114.389738
std	76.593132	13.019695	6.771869	9.116089	249.502091
min	1.000000	1.000000	1.000000	1.000000	0.000000
25%	11.000000	2.000000	1.000000	1.000000	14.000000
50%	23.000000	3.000000	1.000000	2.000000	49.000000
75%	46.000000	7.000000	3.000000	4.000000	119.000000
max	3442.000000	470.000000	165.000000	402.000000	8441.000000

	v	l	d	i	e
count	10885.000000	10885.000000	10885.000000	10885.000000	1.088500e+04
mean	673.758017	0.135335	14.177237	29.439544	3.683637e+04
std	1938.856196	0.160538	18.709900	34.418313	4.343678e+05
min	0.000000	0.000000	0.000000	0.000000	0.000000e+00
25%	48.430000	0.030000	3.000000	11.860000	1.619400e+02
50%	217.130000	0.080000	9.090000	21.930000	2.031020e+03
75%	621.480000	0.160000	18.900000	36.780000	1.141643e+04
max	80843.080000	1.300000	418.200000	569.780000	3.107978e+07

	b	t	lOCode	lOComment	lOBlank
count	10885.000000	1.088500e+04	10885.000000	10885.000000	10885.00000
mean	0.224766	2.046465e+03	26.252274	2.737529	4.62554
std	0.646488	2.413154e+04	59.611201	9.080688	9.96813
min	0.000000	0.000000e+00	0.000000	0.000000	0.00000
25%	0.020000	9.000000e+00	4.000000	0.000000	0.00000
50%	0.070000	1.128300e+02	13.000000	0.000000	2.00000
75%	0.210000	6.342500e+02	28.000000	2.000000	5.00000
max	26.950000	1.726655e+06	2824.000000	344.000000	447.00000

	locCodeAndComment	uniq_Op	uniq_Opnd	total_Op
count	10885.000000	10880.000000	10880.000000	10880.000000
mean	0.370785	11.177592	16.751857	68.110588
std	1.907969	10.085255	26.667883	151.513836
min	0.000000	0.000000	0.000000	0.000000
25%	0.000000	5.000000	4.000000	8.000000
50%	0.000000	11.000000	11.000000	29.000000
75%	0.000000	16.000000	21.000000	71.000000
max	108.000000	411.000000	1026.000000	5420.000000

	total_Opnd	branchCount
count	10880.000000	10880.000000
mean	46.380980	11.292316
std	100.351845	22.597617
min	0.000000	1.000000
25%	6.000000	3.000000
50%	19.000000	5.000000
75%	48.000000	13.000000
max	3021.000000	826.000000

Categorical Feature Summary:

	defects
count	10885
unique	2
top	false
freq	8779

Fig. 2. Summary Statistic

A correlation matrix is a table that shows the pairwise correlation coefficients between variables in the dataset. The correlation coefficient is between -1 to 1, where a correlation value of 1 indicates a perfect positive relationship, 0 denotes no linear relationship, and -1 indicates a perfect negative relationship between variables. The heatmap visualizes these

correlation coefficients, where the color intensity signifies the strength of the correlation. Strong positive correlations are shown by bright red, strong negative correlations by bright blue, and weak correlations by lighter colors. By examining the correlation matrix, it is easy to identify which metrics have strong linear relationships with each other. This information is useful for feature selection, multicollinearity analysis, and understanding the dataset's underlying structure.

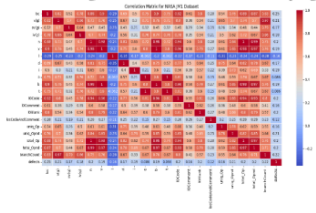


Fig. 3. Correlation Matrix of JM1 Dataset

To complement correlation analysis, feature importance was computed using a Random Forest classifier to identify the most influential software metrics. Top-ranked features included loc, branchCount, uniq_Opnd, uniq_Op, and v(g), all reflecting code complexity. These findings align with the understanding that more complex code tends to be more defect-prone. The selected features were later used to reduce dimensionality and enhance SVM performance in the hybrid model.

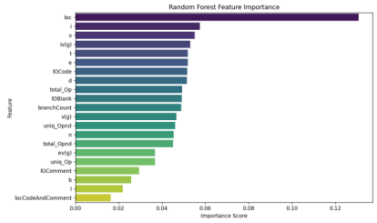


Fig. 4. Random Forest Feature Importance

The class distribution analysis for the JM1 dataset reveals a significant class imbalance. Out of a total of 10,885 samples, 8,779 instances (≈80.7%) are labeled as non-defective (0), while only 2,106 instances (≈19.3%) are labeled as defective (1). This results in an imbalance ratio of approximately 4.2:1, indicating that non-defective samples vastly outnumber the defective ones. Additionally, statistical measures of the defects column confirm this skew, with a mean of 0.193, implying that fewer than 1 in 5 samples are defective. Such imbalance can negatively impact model performance, especially in detecting minority class instances, and therefore justifies the use of resampling techniques like SMOTE or applying class weights during model training.

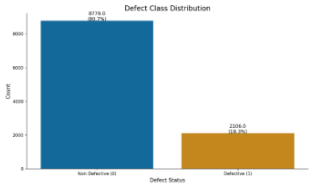


Fig. 5. JM1 Class Distribution

C. DATA PREPROCESSING

The JM1 dataset, obtained from a public GitHub repository in ARFF format, required several preprocessing steps to prepare it for machine learning. Originally developed by NASA under the MDP initiative, the dataset contains software metrics based on McCabe and Halstead complexity measures, along with a binary label indicating defect presence.

The class label, initially stored as byte strings (e.g., b'true', b'false'), was decoded and mapped to numeric values (1 for defective, 0 for non-defective) to ensure compatibility with scikit-learn. The data was then checked for missing values, but none were found, and since all features were numerical, no categorical encoding was necessary.

Features and labels were separated into X and y, respectively, and the dataset was scaled using a RobustScaler to reduce the impact of outliers—particularly beneficial for SVM performance. While minimal cleaning was needed, these preprocessing steps ensured consistency and readiness for modeling.

D. HYBRID MODEL DEVELOPMENT

To evaluate and improve software defect prediction performance, four model configurations were developed and compared. The first two served as baselines using individual classifiers: RF and SVM. The remaining two configurations implemented hybrid models that combined their strengths. A public version of the complete implementation, including preprocessing, model training, and evaluation scripts, has been uploaded to GitHub for transparency and reproducibility.

- Model 1 - RF only

  Trained with 50 estimators to capture non-linear patterns and provide feature importance scores.

- Model 2 – SVM only

  Utilized rbf kernel, sensitive to input scale, to capture complex boundaries.

- Model 3 – RF-SVM Hybrid Model

  Combined predictions from the RF and SVM models using a voting mechanism to balance RF's generalization with SVM's precision.

- Model 4 – RF-SVM+ (Enhanced Hybrid Model)

  Extended the hybrid by stacking RF and SVM outputs (probabilities) as meta-features, fed into a logistic regression meta-classifier. Additionally, Platt scaling was applied to calibrate probabilities, and threshold tuning based on precision-recall curves was used for optimal classification

E. VALIDATION AND VERIFICATION

To ensure both performance and implementation correctness, this study applied validation and verification methods throughout the modeling process. Validation was carried out using an 80:20 train-test split to evaluate model generalization. Key metrics of Precision, Recall, F1-score, and ROC-AUC were computed on the hold-out set to assess the model's effectiveness, particularly in identifying defective modules. Threshold tuning via precision-recall curves was also applied to optimize the decision boundary for better balance between detection and reliability.

Verification focused on the correctness of preprocessing and model integration. Each component was tested in isolation before being assembled into the full pipeline. Intermediate outputs (e.g., decoded labels, scaled features, prediction probabilities) were manually validated. To ensure reproducibility, fixed random seeds were used, and the model architecture was modularized across separate scripts for each classifier, supporting traceability and auditing.

IV. DATA ANALYSIS AND RESULTS

This chapter presents the evaluation and comparison of four models—RF, SVM, RF-SVM, and RF-SVM+—developed for software defect prediction using the JM1 dataset. All models were trained on the same preprocessed data and evaluated using an 80:20 hold-out split. Key metrics include Accuracy, Precision, Recall, F1-score, ROC-AUC, and average runtime.

A. Model Evaluation Results

Each model was executed five times to ensure consistency in runtime, with mean values reported. Across the evaluated models, the RF model demonstrated a moderate balance in prediction metrics, while the SVM model showed strong precision for non-defective modules (Class 0) but failed to effectively capture defective ones (Class 1) due to poor recall. The RF-SVM hybrid provided some improvement in recall by combining ensemble-based generalization with margin-based classification, yet it lacked refined probability calibration. In contrast, the enhanced RF-SVM+ model outperformed all others, delivering superior predictive performance and the highest computational efficiency.

TABLE I. COMPARISON OF PERFORMANCE METRICS IN CLASS 0

Model	Precision	Recall	F1-Score
RF	0.8425	0.9431	0.8900
SVM	0.8112	0.9949	0.8937
RF-SVM	0.8305	0.9727	0.8960
RF-SVM+	0.8414	0.8316	0.8444

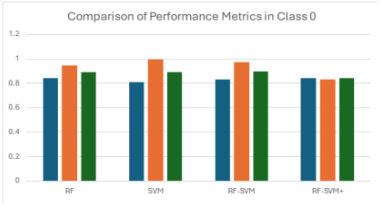


Fig. 6. Comparison of Performance Metrics in Class 0

TABLE II. COMPARISON OF PERFORMANCE METRICS IN CLASS 1

Model	Precision	Recall	F1-Score
RF	0.5215	0.2601	0.3471
SVM	0.5714	0.0286	0.0545
RF-SVM	0.5932	0.1671	0.2607
RF-SVM+	0.8414	0.8316	0.8364

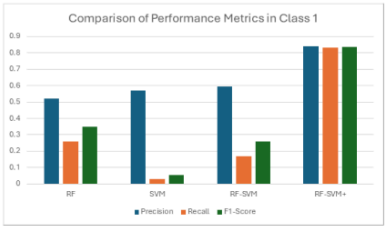


Fig. 7. Comparison of Performance Metrics in Class 1

B. Accuracy and ROC-AUC

In terms of overall accuracy in Table III, RF-SVM+ achieved the highest score of 0.8405, slightly outperforming the other models. However, ROC-AUC is a more reliable metric for imbalanced classification problems like this one. RF-SVM+ again stood out with a ROC-AUC of 0.9096, far surpassing the others, as shown in Table III. This indicates that RF-SVM+ had the best overall capability to distinguish between defective and non-defective modules, even with the class imbalance.

TABLE III. ACCURACY AND ROC-AUC COMPARISON OF CLASSIFICATION MODELS

Model	Accuracy	ROC-AUC
RF	0.8117	0.7546
SVM	0.8089	0.6160
RF-SVM	0.8176	0.7540
RF-SVM+	0.8405	0.9096

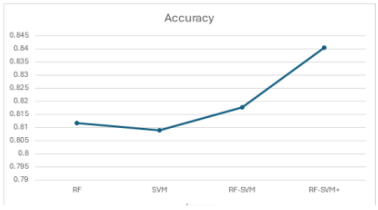


Fig. 8. Accuracy Comparison of Classification Models

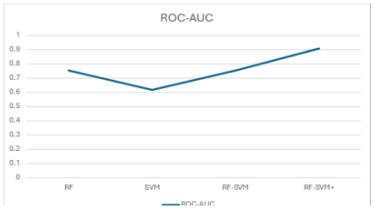


Fig. 9. ROC-AUC Comparison of Classification Models

C. *Average Execution Time*

Execution time was also measured to evaluate model efficiency. While the SVM and RF-SVM models had the longest runtimes (12.3s and 11.364s respectively), the RF-SVM+ model was the fastest, completing in just 0.802s. This is especially significant considering RF-SVM+ also achieved the best classification performance. It suggests the model is not only accurate but also computationally efficient. a sentence.

TABLE IV. AVERAGE EXECUTION TIME

Model	Average Execution Time
RF	1.396s
SVM	12.3s
RF-SVM	11.364s
RF-SVM+	0.802s

Fig. 10. Example of a figure caption. (*figure caption*)

V. DISCUSSION

This chapter analyzed the results of four models, RF, SVM, RF-SVM, and RF-SVM+—with the proposed RF-SVM+ model consistently outperforming the others in all evaluation metrics. It achieved the highest F1-score (0.8364), ROC-AUC (0.9096), and accuracy (0.8405) while maintaining the fastest runtime (0.802s). Its strength lies in combining ensemble learning, calibrated probabilities, and feature selection, making it both precise and efficient for defect prediction. SVM underperformed on imbalanced data due to poor recall. RF provided general reliability but lacked fine-grained precision. The basic RF-SVM hybrid improved recall slightly, while RF-SVM+ introduced meta-model stacking and calibration to achieve superior performance. The model excelled in Class 1 prediction and runtime stability across repeated trials, making it practical for real-world applications like automated QA pipelines. Meanwhile, using only the noisy JM1 dataset may limit generalizability. Other datasets (e.g., CM1, KC1) showed higher performance in preliminary tests. The model lacks testing on cross-project data and excludes semantic/process-level features. In conclusion, RF-SVM+ demonstrates the power of hybrid ML models in improving software defect prediction. While promising, further validation and generalization are needed in broader contexts.

VI. CONCLUSION AND RECOMMENDATIONS

This study developed a hybrid RF-SVM+ model for software defect prediction, integrating feature selection, classification, and meta-learning. Tested on the imbalanced JM1 dataset, it achieved high performance (F1: 0.8364, ROC-AUC: 0.9096) with minimal runtime, outperforming baseline models. The results show that hybridization and techniques like SMOTE and Platt Scaling effectively enhance defect detection accuracy and efficiency. Future work should test the model on multiple datasets (e.g., CM1, KC1), explore advanced resampling methods like ADASYN, apply SHAP for interpretability, incorporate hyperparameter tuning, and assess deployment feasibility in real-world CI/CD pipelines.

ACKNOWLEDGMENT

I would like to express my heartfelt appreciation to all those who supported me throughout this research project. My deepest gratitude goes to my supervisor, Ms. Venushini A/P Rajendran, for her invaluable guidance, encouragement, and insightful feedback that shaped the direction of this work. I also thank Multimedia University for providing the resources and a supportive learning environment, and my lecturers for equipping me with essential skills and knowledge. A special thanks to my family, your unwavering love and support have been my greatest motivation. To everyone who contributed, directly or indirectly, thank you. This project was made possible because of you.

AVAILABILITY OF DATA AND MATERIALS

The source code and implementation files for this project can be downloaded at: https://github.com/squanchyyyy/Hybrid-RF-SVM-Model-for-SDP

The JM1 dataset used in this study is publicly available at: https://github.com/ApoorvaKrisna/NASA-promise-dataset-repository/blob/main/jm1.arff

REFERENCES

[1] Wang, H., Arasteh, B., Arasteh, K., Soleimanian Gharehchopogh, F., & Rouhi, A. (2024). A software defect prediction method using binary gray wolf optimizer and machine learning algorithms. Journal of Software: Evolution and Process, 36(3), 123-135. Retrieved from https://doi.org/10.1016/j.software.2024.01.005

[2] Akhilesh, G. V. D., Raju, N. R. B., Nihaal, R. S., & Amiripalli, S. S. (2021). A study on machine learning techniques for predicting software defects. International Research Journal of Engineering and Technology (IRJET), 8(12), 1014. Retrieved from https://www.researchgate.net/publication/357367579

[3] Mishra, A., & Sharma, A. (2024). Deep learning-based continuous integration and continuous delivery software defect prediction with

effective optimization strategy. Knowledge-Based Systems, 284, 111835. https://doi.org/10.1016/j.knosys.2024.111835

[4] Feng, S., Keung, J., Yu, X., Xiao, Y., Bennin, K. E., Kabir, M. A., & Zhang, M. (2020). COSTE: Complexity-based oversampling technique to alleviate the class imbalance problem in software defect prediction. Information and Software Technology, 127, 106368. https://doi.org/10.1016/j.infsof.2020.106368

[5] Dong, X., Liang, Y., Miyamoto, S., & Yamaguchi, S. (2023). Ensemble learning-based software defect prediction. ICT Express, 10(1), 80-85. https://doi.org/10.1016/j.icte.2023.10.004

[6] Sharma, T., Jatain, A., Bhaskar, S., & Pabreja, K. (2023). Ensemble machine learning paradigms in software defect prediction. In Proceedings of the International Conference on Machine Learning and Data Engineering. https://doi.org/10.1016/SD187705092300029

[7] Stradowski, S., & Madeyski, L. (2023). Industrial applications of software defect prediction using machine learning: A business-driven systematic literature review. Information and Software Technology, 157, 107177. https://doi.org/10.1016/j.infsof.2023.107177

[8] Stradowski, S., & Madeyski, L. (2022). Machine learning in software defect prediction: A business-driven systematic mapping study. Information and Software Technology, 152, 107069. https://doi.org/10.1016/j.infsof.2022.107069

[9] Saharudin, S. N. A., Wei, K. T., & Na, K. S. (2020). Machine learning techniques for software bug prediction: A systematic review. ResearchGate. Retrieved from https://www.researchgate.net/publication/346022956

[10] Aquil, M. A. I., & Ishak, W. H. W. (2020). Predicting software defects using machine learning techniques. International Journal of Advanced Trends in Computer Science and Engineering, 9(4), 3529-3535. https://doi.org/10.30534/ijatcse/2020/352942020

[11] Santos, G., Figueiredo, E., Veloso, A., Viggiato, M., & Ziviani, N. (2021). Predicting software defects with explainable machine learning. ResearchGate. Retrieved from https://www.researchgate.net/publication/349868544

[12] Al-Smadi, Y., Eshtay, M., Al-Qerem, A., Nashwan, S., Ouda, O., & Abd El-Aziz, A. A. (2023). Reliable prediction of software defects using Shapley interpretable machine learning models. Ain Shams Engineering Journal, 14(6), 101989. https://doi.org/10.1016/j.asej.2023.101989

[13] Ibrahim, A. M., Abdelsalam, H., & Taj-Eddin, I. A. T. F. (2023). Software defect prediction at method level using ensemble learning techniques. International Journal of Intelligent Computing and Information Sciences, 23(2), 28-49. https://doi.org/10.21608/ijicis.2023.189934.1251

[14] Chayadevi, K., & Reddy, C. S. (2023). Software defect prediction using machine learning algorithms. Journal of Emerging Technologies and Innovative Research (JETIR), 10(9), e672. Retrieved from https://www.jetir.org/view?paper=JETIR2309482

[15] Jumare, M., Haruna, & Chinyio, D. T. (2024). Software defect prediction using machine learning and deep learning techniques. Kasu Journal of Computer Science, 1(3), 527-543. https://doi.org/10.47514/kjcs/2024.1.3.0010

[16] Kethireddy, J., Aravind, E., & Vijayakamal, M. (2023, May). Software defects prediction using machine learning algorithms. Conference Paper, ResearchGate. Retrieved from https://www.researchgate.net/publication/370496703

[17] Batool, I., & Khan, T. A. (2022). Software fault prediction using data mining, machine learning, and deep learning techniques: A systematic literature review. Computers and Electrical Engineering, 100, 107886. https://doi.org/10.1016/j.compeleceng.2022.107886

[18] Begum, M., Shuvo, M. H., Ashraf, I., Mamun, A. A., Uddin, J., & Samad, M. A. (2023). Software defects identification: Results using machine learning and explainable artificial intelligence techniques. IEEE Access, 11, 124567–124580. https://doi.org/10.1109/ACCESS.2023.3329051

[19] Thota, M. K., Shajin, F. H., & Rajesh, P. (2020). Survey on software defect prediction techniques. International Journal of Applied Science and Engineering, 17(4), 331–345. https://doi.org/10.6703/IJASE.202012_17(4).331

[20] Khalid, A., Badshah, G., Ayub, N., Shiraz, M., & Ghouse, M. (2023). Software defect prediction analysis using machine learning techniques. Sustainability, 15(6), 5517. https://doi.org/10.3390/su15065517

[21] Esteves, G., Figueiredo, E., Veloso, A., Viggiato, M., & Ziviani, N. (2020). Understanding machine learning software defect predictions. Automated Software Engineering, 27(3-4), 369–392. https://doi.org/10.1007/s10515-020-00277-4

[22] Matloob, F., Ghazal, T. M., Taleb, N., Aftab, S., Ahmad, M., Khan, M. A., Abbas, S., & Soomro, T. R. (2021). Software defect prediction using ensemble learning: A systematic literature review. IEEE Access, 9, 97279–97298. https://doi.org/10.1109/ACCESS.2021.3095559

[23] Singh, M., & Chhabra, J. K. (2024). Machine learning based improved cross-project software defect prediction using new structural features in object-oriented software. Applied Soft Computing, 165, 112082. https://doi.org/10.1016/j.asoc.2024.112082

[24] Abbas, S., Aftab, S., Khan, M. A., Ghazal, T. M., Al Hamadi, H., & Yeun, C. Y. (2023). Data and ensemble machine learning fusion based intelligent software defect prediction system. Computers, Materials & Continua. https://doi.org/10.32604/cmc.2023.037933

[25] Fan, X., Mao, J., Lian, L., Yu, L., Zheng, W., & Ge, Y. (2024). Software defect prediction method based on stable learning. Computers, Materials & Continua. https://doi.org/10.1016/j.cmc.2024.1966

[26] Sahni, K., & Chandra, G. (2021). A review on software defect prediction using machine learning techniques. International Conference on Intelligent Technologies & Science (ICITS-2021), International Journal of Research and Development in Applied Science and Engineering. Retrieved from https://ijrdase.com/ijrdase/wp-content/uploads/2021/07/A-Review-on-Software-Defect-Prediction-Using-Machine-Learning-Techniques-Karishma.pdf

[27] Gautam, S., Khunteta, A., & Ghosh, D. (2024). Comparison of machine learning models for effective software fault detection. International Journal of Intelligent Systems and Applications in Engineering, 12(21s), 834–840. Retrieved from https://www.researchgate.net/publication/380266848_Comparison_of_Machine_Learning_Models_for_Effective_Software_Fault_Detection

[28] Shailee, N. M., Alam, A., Ahmed, T., Rudro, R. A. M., & Nur, K. (2024). Software bug prediction using machine learning on JM1 dataset. In Proceedings of the 2024 International Conference on Advances in Computing, Communication, Electrical, and Smart Systems (iCACCESS). IEEE. https://doi.org/10.1109/iCACCESS10499572

[29] Ali, M., Mazhar, T., Arif, Y., Al-Otaibi, S., Ghadi, Y. Y., Shahzad, T., Khan, M. A., & Hamam, H. (2024). Software defect prediction using an intelligent ensemble-based model. IEEE Access, 12. https://doi.org/10.1109/ACCESS.2024.3358201

[30] Azam, M., Nouman, M., & Gill, A. R. (2024). Comparative analysis of machine learning techniques to improve software defect prediction. KJCSIS, 5(2). https://doi.org/10.51153/kjcis.v5i2.96

241UC2414K_LEW_ENG_JEAN_REVIEW_PAPER.docx

ORIGINALITY REPORT

12%	8%	11%	%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS

PRIMARY SOURCES

1	trepo.tuni.fi Internet Source	2%
2	Raghunath Dey, Jayashree Piri, Biswaranjan Acharya, Pragyan Paramita Das, Vassilis C. Gerogiannis, Andreas Kanavos. "A Hybrid Evolutionary Fuzzy Ensemble Approach for Accurate Software Defect Prediction", Mathematics, 2025 Publication	1%
3	Babatunde Oremule, Gabrielle H. Saunders, Karolina Kulk, Alexander d'Elia, Iain A. Bruce. "Understanding, experience, and attitudes towards artificial intelligence technologies for clinical decision support in hearing health: a mixed-methods survey of healthcare professionals in the UK", The Journal of Laryngology & Otology, 2024 Publication	1%
4	arxiv.org Internet Source	1%
5	www.scirp.org Internet Source	1%
6	journals.gaftim.com Internet Source	1%
7	Mehrasa Modanlou Jouybari, Alireza Tajary, Mansoor Fateh, Vahid Abolghasemi. "A novel deep neural network structure for software fault prediction", PeerJ Computer Science, 2024	<1%

	Publication	
8	www.researchgate.net Internet Source	<1%
9	www.technologyshout.com Internet Source	<1%
10	ela.kpi.ua Internet Source	<1%
11	kinetik.umm.ac.id Internet Source	<1%
12	pubmed.ncbi.nlm.nih.gov Internet Source	<1%
13	Zhao Tian, Xuan Zhao, Xiwang Li, Xiaoping Ma, Yinghao Li, Youwei Wang. "Multi-modal semantics fusion model for domain relation extraction via information bottleneck", Expert Systems with Applications, 2024 Publication	<1%
14	ieeemy.org Internet Source	<1%
15	ijrpr.com Internet Source	<1%
16	www.mdpi.com Internet Source	<1%
17	Maria Naqvi, Fredrik Fineide, Tor Paaske Utheim, Colin Charnock. "Culture- and non-culture-based approaches reveal unique features of the ocular microbiome in dry eye patients", The Ocular Surface, 2024 Publication	<1%
18	Susmita Haldar, Luiz Fernando Capretz. "Interpretable Software Defect Prediction from Project Effort and Static Code Metrics", Computers, 2024 Publication	<1%

19	Yanyang Zhao, Yawen Wang, Dalin Zhang, Yunzhan Gong. " Eliminating the high false-positive rate in defect prediction through with adjustable weight ", Expert Systems, 2022 Publication	<1 %
20	Adina Turcu-Stiolica, Claudiu Marinel Ionele, Bogdan Silviu Ungureanu, Mihaela-Simona Subtirelu. "The Effects of Arginine-Based Supplements on Fatigue Levels following COVID-19 Infection: A Prospective Study in Romania", Healthcare, 2023 Publication	<1 %
21	S. Kannadhasan, R. Nagarajan, Alagar Karthick, V. Kumar Chinnaiyan. "Technological Applications for Smart Sensors", Apple Academic Press, 2025 Publication	<1 %
22	Yu Tang, Qi Dai, Ye Du, Lifang Chen, Xuanwen Niu. "A software defect prediction method based on learnable three-line hybrid feature fusion", Expert Systems with Applications, 2023 Publication	<1 %
23	studentsrepo.um.edu.my Internet Source	<1 %
24	thesai.org Internet Source	<1 %
25	www.nature.com Internet Source	<1 %
26	Manpreet Singh, Jitender Kumar Chhabra. "Machine learning based improved cross-project software defect prediction using new structural features in object oriented software", Applied Soft Computing, 2024 Publication	<1 %

27	Mona Nashaat, James Miller. "Refining Software Defect Prediction through Attentive Neural Models for Code Understanding", Journal of Systems and Software, 2024 Publication	<1%
28	Akshat Pandey, Akshay Jadhav. "Towards Effective Software Defect Prediction Using Machine Learning Techniques", SN Computer Science, 2024 Publication	<1%
29	Debbie Hahs-Vaughn, Richard Lomax. "An Introduction to Statistical Concepts - Third Edition", Routledge, 2019 Publication	<1%
30	Thangaprakash Sengodan, Sanjay Misra, M Murugappan. "Advances in Electrical and Computer Technologies", CRC Press, 2025 Publication	<1%

Exclude quotes	On	Exclude matches	Off
Exclude bibliography	On		